

Chapter 3

Preloading

Queues, caches, loading screens - and files that go missing



Today

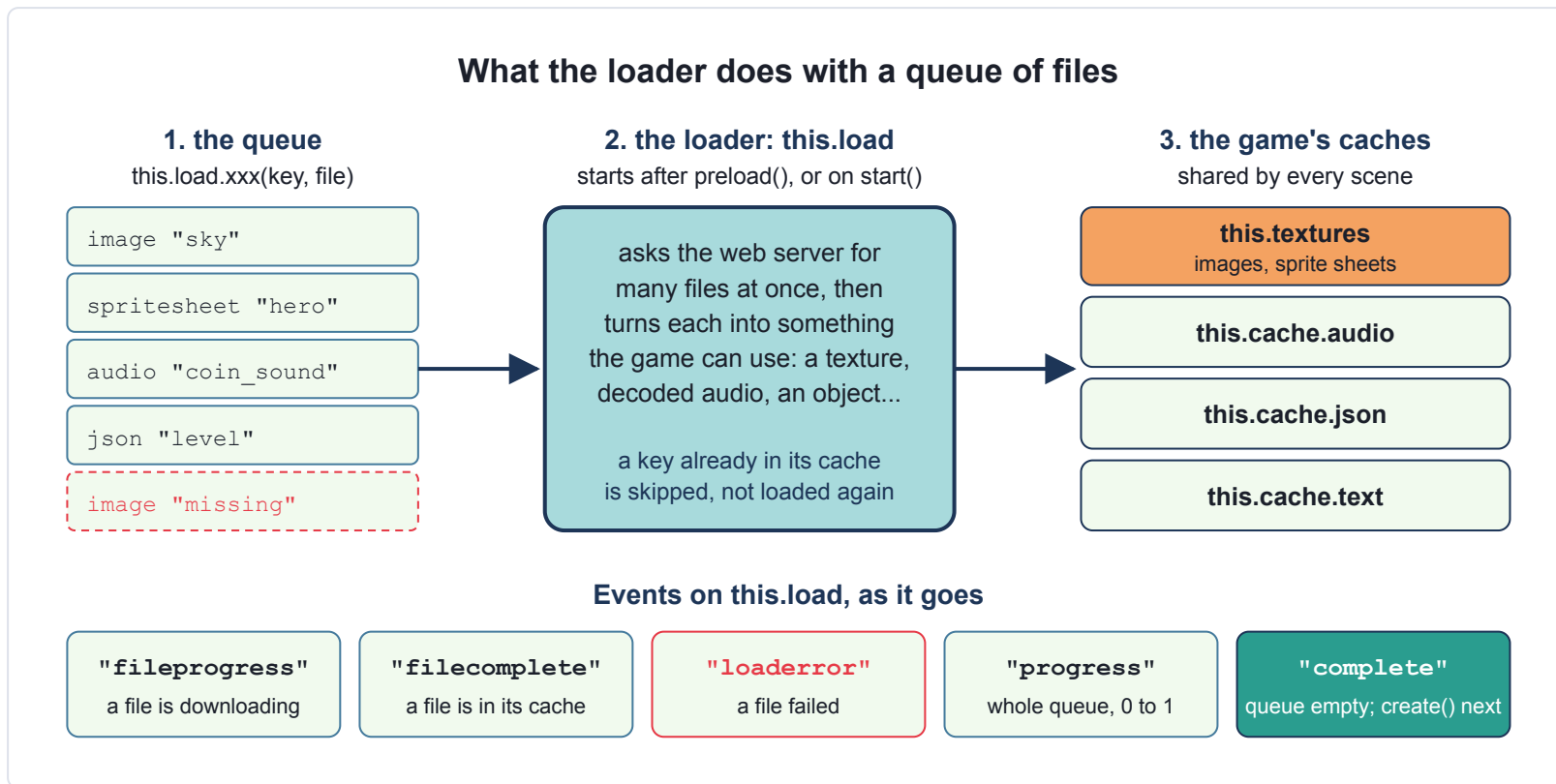
- what `this.load.image(...)` **really** does
- keys, and the game's **caches**
- **Boot -> Preload -> Menu**, with a progress bar
- every kind of file: image, sprite sheet, audio, JSON, text, pack
- when a file is missing
- loading in the middle of a game

preload() only makes a list

```
// preload() - queued, NOT loaded  
this.load.image(CRATE_KEY, CRATE_FILE);  
  
// create() (in showImages) - loaded by now  
this.add.image(x + 70, y + 120, CRATE_KEY).setScale(2);
```

1. `preload()` queues files
2. Phaser starts the loader when `preload()` **returns**
3. `create()` runs when the queue is **empty**

Queue -> loader -> caches



Keys and caches

Loaded with	Kept in
image , spritesheet	this.textures
audio	this.cache.audio
json	this.cache.json
text	this.cache.text

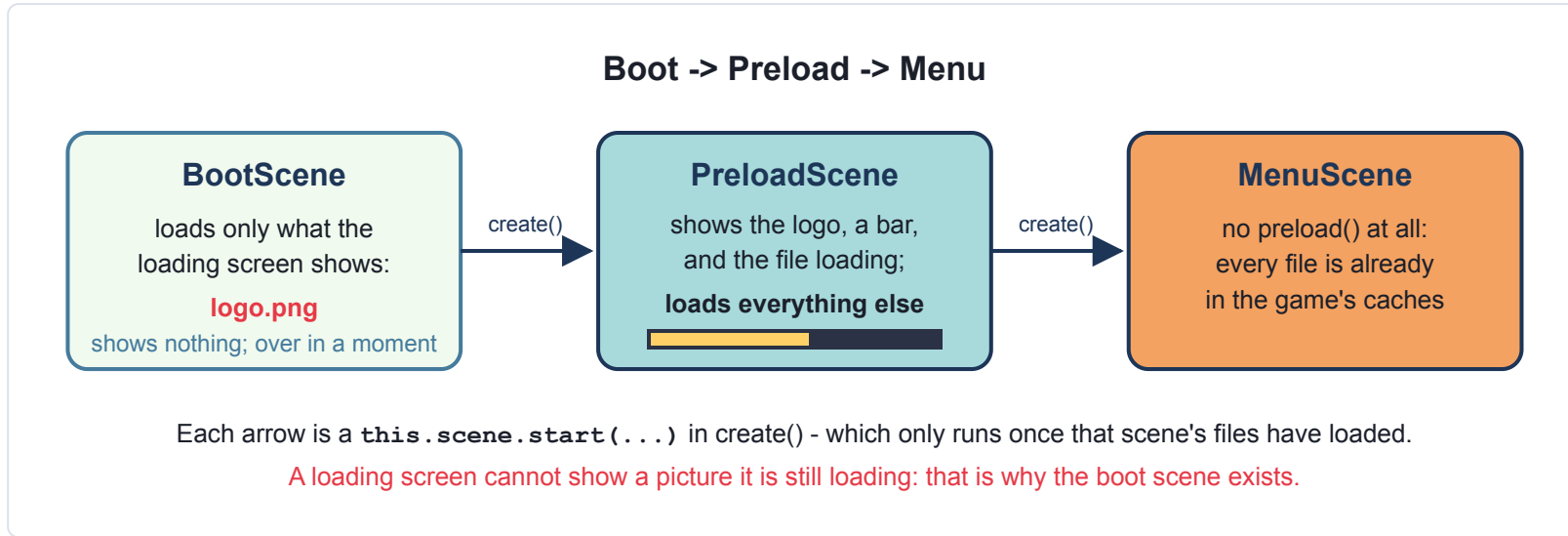
- the caches belong to the **game** - every scene can use them
- a key is unique **within its kind**
- a key that is already loaded is **skipped**

Paths

```
this.load.setPath("assets/images/");  
this.load.image(SKY_KEY, "sky.png");  
this.load.image(SPACE_KEY, "space.png");  
  
this.load.setPath("assets/audio/");  
this.load.audio(COIN_SOUND_KEY, "coin.wav");
```

- address = **base URL + path + file name**
- relative to the **page**: `public/` is copied to `dist/`
- no leading `/`; capitals matter on a web server

Boot -> Preload -> Menu



A loading screen cannot show a picture it is still loading.

The boot scene

```
preload(): void {  
    this.load.image(LOGO_KEY, LOGO_FILE);  
}  
  
// create() only runs once everything queued in preload() has loaded  
create(): void {  
    this.scene.start(PRELOAD_SCENE);  
}
```

Small, fast, shows nothing.

Drawing the bar: Graphics

```
const box = this.add.graphics();
box.fillStyle(BOX_COLOUR);
box.fillRect(barX - BOX_PADDING, BAR_Y - BOX_PADDING,
  BAR_WIDTH + BOX_PADDING * 2, BAR_HEIGHT + BOX_PADDING * 2);

const bar = this.add.graphics();
```

- `fillStyle(0x2b3245)` - colours as hex numbers
- `fillRect(x, y, width, height)` - from the **top-left**
- `clear()` - rub it all out

Listening to the loader

```
this.load.on("progress", (value: number) => {  
    bar.clear();  
    bar.fillStyle(BAR_COLOUR);  
    bar.fillRect(barX, BAR_Y, BAR_WIDTH * value, BAR_HEIGHT);  
});  
  
this.load.on("fileprogress", (file: Phaser.Loader.File) => {  
    fileText.setText(`Loading ${file.src}`);  
});  
  
this.load.on("complete", () => {  
    fileText.setText("All done!");  
});
```

Also: "start", "filecomplete", "loaderror"

Try it: Loading Screen

- build and run `ch03_loading_screen` - a flash
- F12 -> **Network** -> **Disable cache**, **Fast 4G** -> refresh
- set `COPIES` to 0 - what changes?
- press **R** in the menu: "Loaded **0** files" - why?



Every kind of file

```
this.load.image(CRATE_KEY, CRATE_FILE);  
this.load.spritesheet(HERO_KEY, HERO_FILE, { frameWidth:  
this.load.audio(POP_KEY, POP_FILE);  
this.load.json(LEVEL_KEY, LEVEL_FILE);  
this.load.text(NOTES_KEY, NOTES_FILE);  
this.load.pack(PACK_KEY, PACK_FILE, PACK_SECTION);
```



Sprite sheets

- one picture, cut into equal **frames**, numbered from 0
- `coin_spin.png` is 192 x 32 -> six 32 x 32 frames
- the frame is the fourth argument:

```
this.add.image(frameX, y + 215, COIN_SPIN_KEY, frame);
```

- wrong frame size = frames cut in the wrong place
- no frame size at all = `Invalid frameWidth given.`

JSON - and saying what is in it

```
export interface LevelData {  
  name: string;  
  player: Point;  
  gems: Point[];  
}  
  
const level: LevelData = this.cache.json.get(LEVEL_KEY);  
level.gems.forEach((gem) => {  
  this.add.image(originX + gem.x, originY + gem.y, GEM_KEY);  
});
```

`get` returns `any`. The interface checks the **code**, never the **file**.

Asset packs

```
{
  "pickups": {
    "path": "assets/",
    "files": [
      { "type": "image", "key": "heart", "url": "images/heart.png" },
      { "type": "image", "key": "coin", "url": "images/coin.png" },
      ...
    ]
  }
}
```

- the list of files moves **out of the code**
- sections; each can set its own `path`
- the pack is a file too - with a key, in the JSON cache

When a file is missing

```
this.load.on("loaderror", (file: Phaser.Loader.File) => {  
    this.failed.push(`${file.key} (${file.src})`);  
});
```

Missing	Result
picture	the missing texture - black, green outline
sound	Audio key "pop" not found in cache
JSON / text	undefined

First stop: the console's red **404** line.

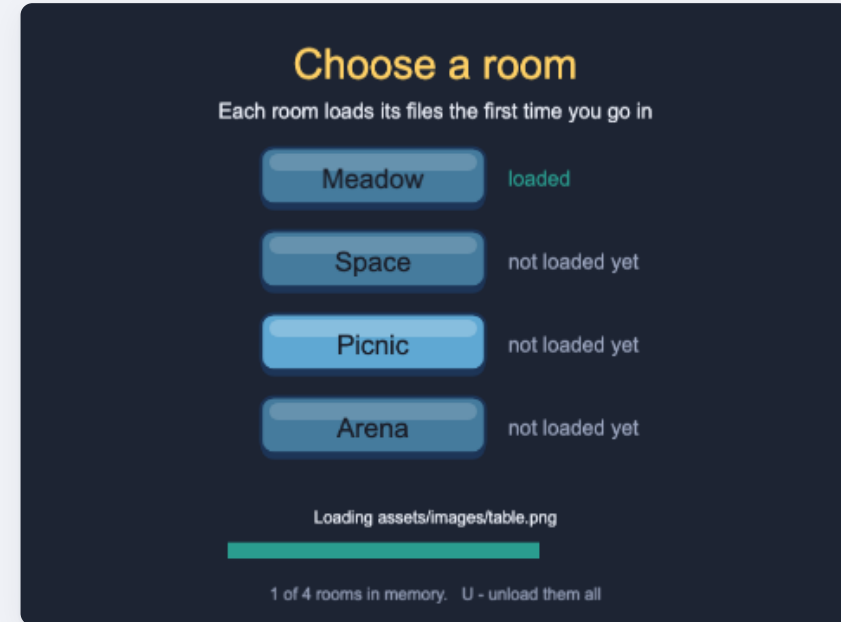
Loading later

```
this.queueRoom(room);  
this.load.once("complete", () => {  
    this.enterRoom(index);  
});  
this.load.start();
```

- outside `preload()` , **you** start the loader
- check first: `this.textures.exists(key)` , `this.cache.audio.exists(key)`
- ignore clicks while `this.load.isLoading()`
- let go: `this.textures.remove(key)` , `this.cache.audio.remove(key)`

Try it: Load on Demand

- throttle the network, choose a room: watch it load
- ESC, choose it again: instant
- Meadow, then Arena: one file fewer - why?
- **U**, then go back in



Summary

- `load...` **queues**; `create()` runs when the queue is empty
- files are kept **by key**, in game-wide caches; loaded keys are skipped
- Boot -> Preload -> Menu; a `Graphics` bar moved by `"progress"`
- image, spritesheet, audio, json (+ interface), text, pack
- missing: `"loaderror"`, the missing texture, the console
- later: queue, `once("complete")`, `this.load.start()`

Challenges

1. **Restyle the bar** - rounded, outlined, red -> orange -> green
2. **Percentage** - whole numbers, never 100% too soon
3. **Hearts in the level** - JSON and interface together
4. **Keep it moving** - a spinner that turns while loading
5. **When files fail** - an error screen, try again or carry on
6. **Rooms from a pack** - every room's files in one asset pack

Next: **Chapter 4 - The life of a scene**